

Как настроить frontend-репозиторий для работы с LLM

Качество ответа языковой модели зависит не только от выбранной модели, но и от того, насколько понятно устроен сам репозиторий. LLM лучше работает, когда правила проекта доступны в явном виде, архитектурные границы стабильны, а команды проверки можно запустить без ручной подготовки окружения.

1. Единая точка входа для инструкций

В корне репозитория полезно разместить файл AGENTS.md. В нем следует кратко описать назначение проекта, основные каталоги, ограничения архитектуры, обязательные команды и критерии готовности задачи. Такой файл понятен разным AI-инструментам и не привязывает команду только к Cursor, Claude Code или другому конкретному клиенту.

Инструкции должны быть проверяемыми. Вместо формулировки «пиши хороший код» лучше указать: использовать TypeScript strict, не обращаться к API напрямую из UI-компонентов, добавлять тест для исправленного сценария и запускать линтер перед завершением задачи.

2. Предсказуемая структура

Модели проще ориентироваться в репозитории, где каталог features содержит бизнес-функции, shared - переиспользуемые элементы, app - сборку приложения, а public - статические файлы. Названия директорий должны отражать назначение, а не историю появления кода. Случайные каталоги вроде temp, misc и common2 создают лишнюю неопределенность.

Контекст, который действительно помогает модели

Не нужно загружать модель всей документацией проекта при каждом запросе. Полезнее дать компактный слой постоянных правил и подключать подробный контекст только для текущей задачи.

3. Документация рядом с кодом

Для каждого крупного модуля стоит хранить README с назначением, публичным API, примерами использования и известными ограничениями. Если компонент зависит от внутреннего UI Kit, зафиксируйте ссылку на документацию, используемые токены и предпочтительные компоненты. Это снижает вероятность того, что модель создаст дублирующую кнопку, модальное окно или собственную систему отступов.

4. Команды как контракт

В `package.json` должны быть стандартные команды: `lint`, `typecheck`, `test`, `test:e2e` и `build`. Их поведение должно совпадать локально и в CI. Когда AI-агент может самостоятельно выполнить проверки и получить однозначный код завершения, количество ложных утверждений о готовности заметно снижается.

Полезно выделить быстрый набор проверок для обычного изменения и полный набор для релиза. Например, `lint` и `typecheck` запускаются после каждого изменения, а `e2e` и `production build` - перед объединением ветки.

5. Примеры вместо длинных запретов

Один эталонный feature-модуль часто полезнее десяти абзацев правил. В нем должны быть показаны работа с API, обработка ошибок, тестирование, локализация и экспорт публичного интерфейса. Модель сможет использовать этот модуль как шаблон и воспроизводить реальные соглашения команды.

Практический минимальный набор

Ниже приведен набор элементов, которого обычно достаточно, чтобы улучшить ответы AI-агентов без сложной инфраструктуры и отдельной векторной базы.

6. Что добавить в репозиторий

AGENTS.md - краткие правила проекта и критерии готовности. README.md - запуск и устройство приложения. docs/architecture - решения, границы модулей и ключевые диаграммы. docs/examples - эталонные реализации. scripts - воспроизводимые команды проверки и генерации. CODEOWNERS - зоны ответственности. Шаблон pull request - список обязательных проверок, рисков и скриншотов.

7. Что не стоит делать

Не следует копировать одинаковые правила в несколько файлов: они быстро расходятся. Не стоит превращать AGENTS.md в энциклопедию на сотни страниц. Не нужно хранить в инструкциях секреты, токены и внутренние данные. Также опасно разрешать агенту выполнять произвольные команды без ограничений и проверки diff.

8. Критерий хорошей настройки

Новая сессия AI-агента должна за несколько минут понять, где находится нужный код, какие ограничения действуют, как проверить результат и что считается завершенной задачей. Если для этого каждый раз требуется вручную пересказывать архитектуру, значит знания проекта еще не зафиксированы в репозитории.

Итог

Начните с AGENTS.md, стандартных команд проверки и одного качественного примера модуля. Затем добавляйте документацию только там, где модель и разработчики регулярно ошибаются. Такой подход остается переносимым между разными LLM и одновременно улучшает обычную инженерную дисциплину команды.